

Experiment 5

Arman Imam

EEE3342C

25Fall 0011

10/13/2025

Equipment:

Computer with Xilinx ISE software, Vivado 2017.4 specifically, and Artix-7 FPGA Board

Objective:

The objective of experiment 5 is to introduce the user to multiplexers and decoders, and how it's applied in Verilog. For part 1, the user is asked to create a 2-1 Mux and 2-4 Decoder based on the truth table given from the experiment, and to use a testbench for test for expected results. For part 2, the user is expected to create a simple 8-to-1 Mux, with 8 specific inputs specified by the experiment, and to use a testbench to verify the waveform, as well as test it on an FPGA board. Part 3 has the same tasks, except the user is creating a simple 3-to-8 Decoder.

Design Steps:

I began by creating a single project file, where I created the specified source files (2-to-1 Mux, 2-to-4 Decoder, 8-to-1 Mux, and 3-to-8 Decoder), and created a specific test bench for each source file. I also created two set of constraints to test the FPGA board for parts 2 and 3.

Part 1:

Verilog RTL of mux

The source code for the 2-1 Multiplexer has 3 inputs(A,B,S), and 1 output(F). A and B are input signals, and S is the selected signal to the output signal. When S is 0, then F = A, and when S is 1, F = B.

```
1  `timescale 1ns / 1ps
2  //2-1 Multiplexer
3  module mux(
4      //A= index 0, B = index 1, S = Index Select
5      input A, B, S,
6      output reg F
7  );
8      always @(*) begin
9          case (S)
10             1'b0: F = A;
11             1'b1: F = B;
12             default: F = 0;
13          endcase
14      end
15  endmodule
```

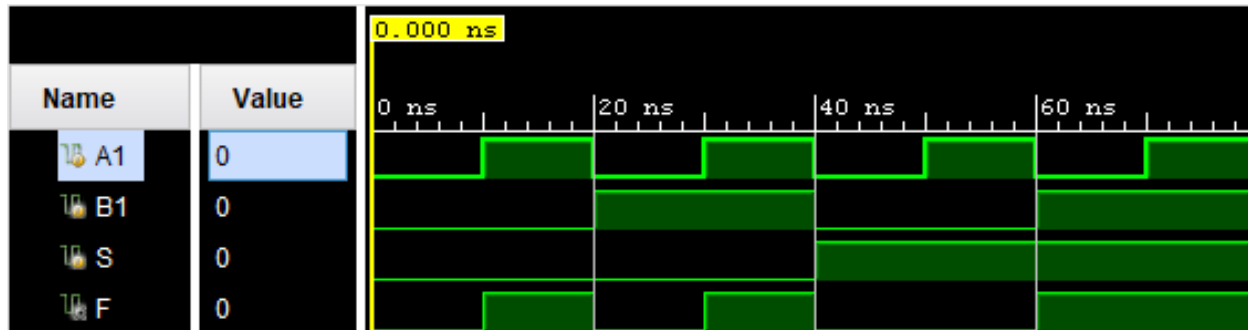
Verilog RTL of decoder

The source code for the 2-to-4 Decoder takes 2 inputs(A, b) and 4 total outputs ([3:0] F). Each part of F is determined by A and B through AND and NOT gates.

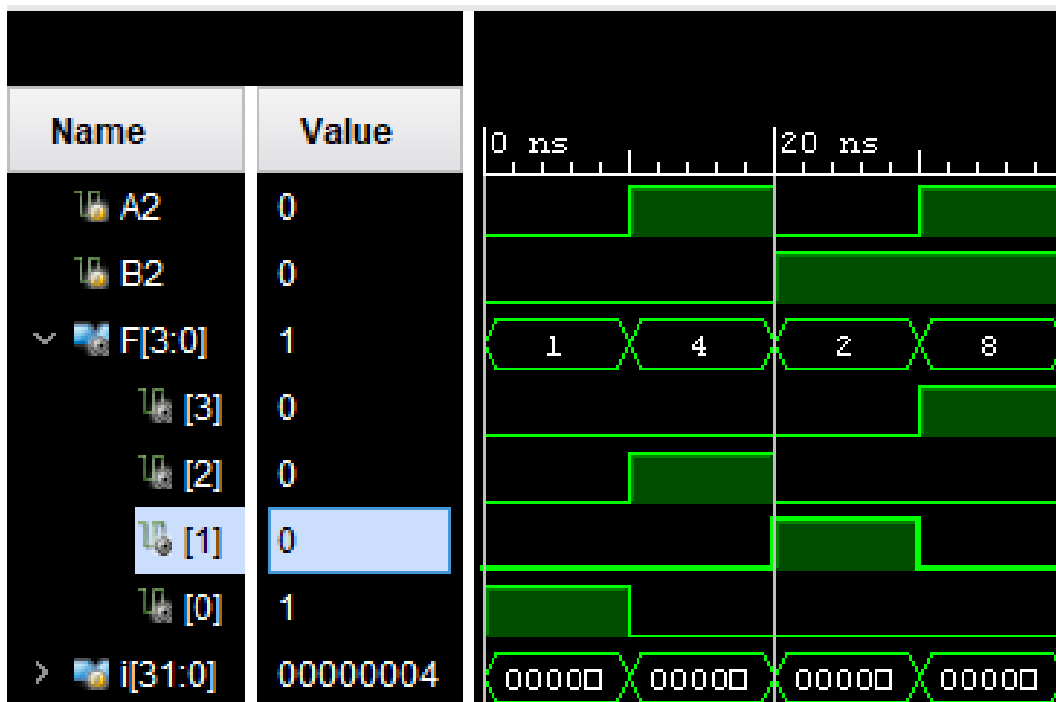
```
1  `timescale 1ns / 1ps
2  //2-4 decoder
3  module decoder_2to4(
4      input A, B,
5      output reg [3:0] F
6  );
7
8      always @(*) begin
9          F[0] = ~A & ~B;
10         F[1] = ~A & B;
11         F[2] = A & ~B;
12         F[3] = A & B;
13     end
14 endmodule
```

Screenshot of waveforms

Mux: This waveform goes through every iteration of A, B, and S, where F is only 1 when {A, B, S} = 100, 110, 011, 111.



Decoder: This waveform goes through every iteration of A and B, where F is only 1 at one point in each iteration. For example, when {A, B} = 00, F[0] = 1, and if {A, B} = 11, F[3] = 1.



Part 2:

Verilog RTL of 8-1 mux

The source code for a modified 8-to-1 Mux has 11 inputs (A, B, C, and [7:0] in) and 1 output (F). The selection lines are A, B, and C, while the input signals are [7:0] in. In a traditional 8-to-1 mux, the binary digits of A, B, C should equal the input signal of that position; ABC = 000 should activate in[0]. The task of part 2 asks for the input signals in this arrangement.

```
1  `timescale 1ns / 1ps
2  //8 to 1 Mux
3  //8 inputs on the left, number of selects = 4, 2^3 = 8
4  module mux_8to1(
5      input A, B, C, // Select Lines
6      input [7:0] in,
7      output reg F
8  );
9      wire [2:0] S = {A, B, C};
10     always @(*) begin
11         case (S)
12             3'b000: F = in[2];
13             3'b001: F = in[6];
14             3'b010: F = in[5];
15             3'b011: F = in[0];
16             3'b100: F = in[1];
17             3'b101: F = in[3];
18             3'b110: F = in[7];
19             3'b111: F = in[4];
20             default: F = 1'b0;
21         endcase
22     end
23
24 endmodule
```

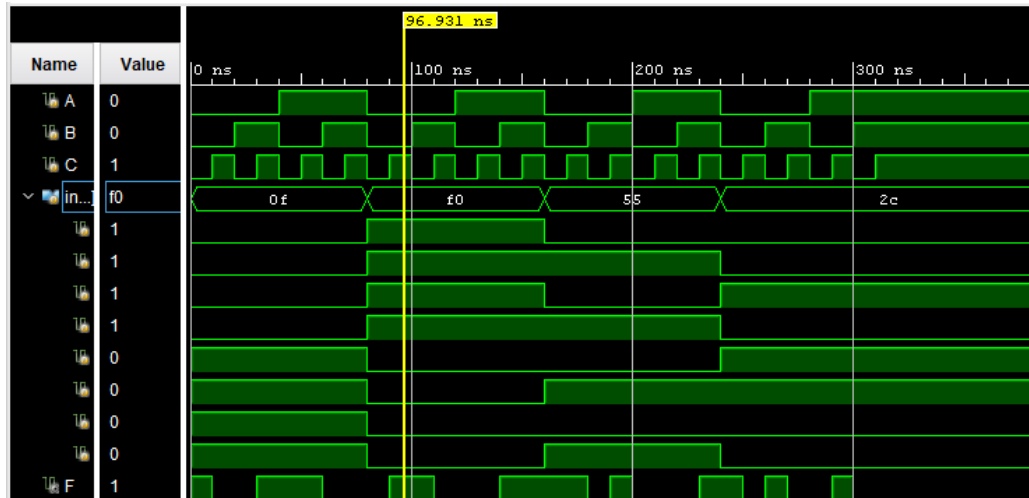
Table of I/O used on FPGA

In the constraints for the Artix-7 board I've specified the inputs and outputs of the board. From right to left on the switches, it goes C, B, A, In[0], In[1]...in[7]. The output light is on the very right.

T2	T3	V2	W13	W14	V15	W15	W17	W16	V16	V17	U16
In[7]	In[6]	In[5]	In[4]	In[3]	In[2]	In[1]	In[0]	A	B	C	F

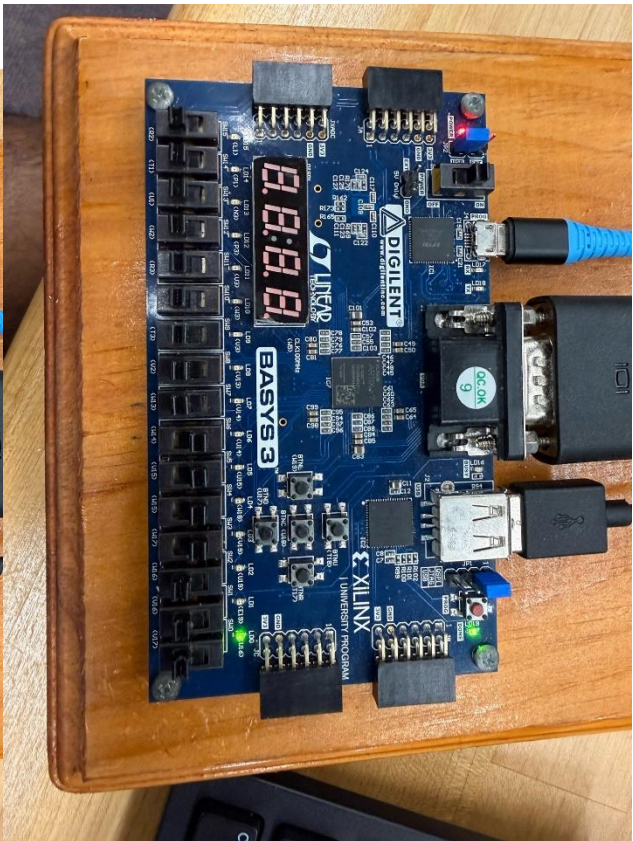
Screenshot of 8-1 mux waveform

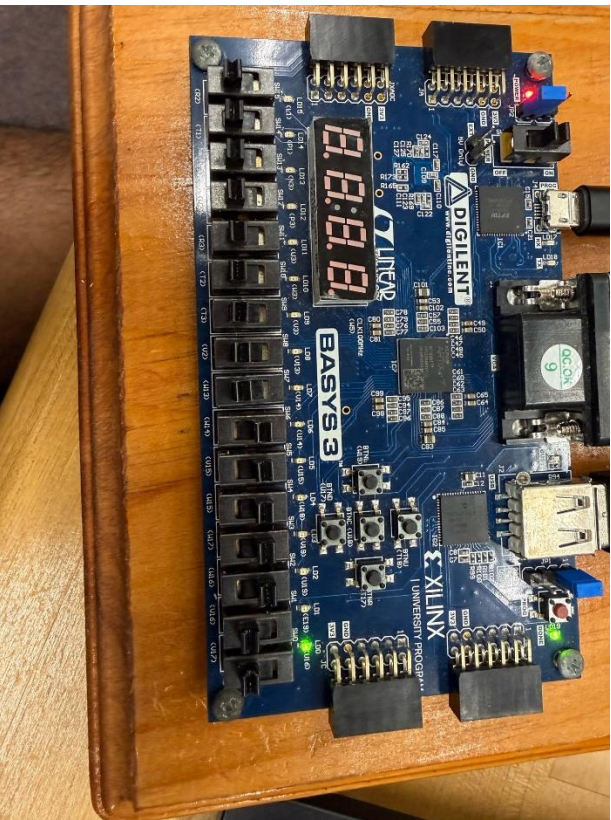
In this waveform, I've tested 4 different sets of inputs. They go in order of 00001111, 11110000, 01010101, and 00101100. In each case, there are 4 instances where F is 1.



Pictures of FPGA board

For the FPGA board, I've only tested for when in = 00001111. There are four instances for when F is 1, which is reflected in both the waveform and FPGA board. In order of {A, B, C} = 000, 011, 100, and 101.





Part 3:

Verilog RTL of 3-8 decoder

The source code for a modified 3-to-8 decoder has 3 inputs (A, B, C) and 8 output ([7:0] F). The order of outputs was given by the experiment brief. I've provided two ways to accomplish the same task (the other is in quotations). In a traditional 3-8 Decoder, F[0] would be $\sim A \& \sim B \& \sim C$, and go through each binary iteration till it reaches F[7], which would be $A \& B \& C$.

```
1  `timescale 1ns / 1ps
2
3
4  module decoder_3to8(
5      input A, B, C,
6      output reg [7:0] F
7  );
8      always @(*)begin
9          F[0] = ~A & B & C;
10         F[1] = A & ~B & ~C;
11         F[2] = ~A & ~B & ~C;
12         F[3] = A & ~B & C;
13         F[4] = A & B & C;
14         F[5] = ~A & B & ~C;
15         F[6] = ~A & ~B & C;
16         F[7] = A & B & ~C;
17     end
18 endmodule
19
20 /*
21     //F = 8'b00000000;
22     case ({A, B, C})
23         3'b000: F[2] = 1;
24         3'b001: F[6] = 1;
25         3'b010: F[5] = 1;
26         3'b011: F[0] = 1;
27         3'b100: F[1] = 1;
28         3'b101: F[3] = 1;
29         3'b110: F[7] = 1;
30         3'b110: F[4] = 1;
31     endcase */
```

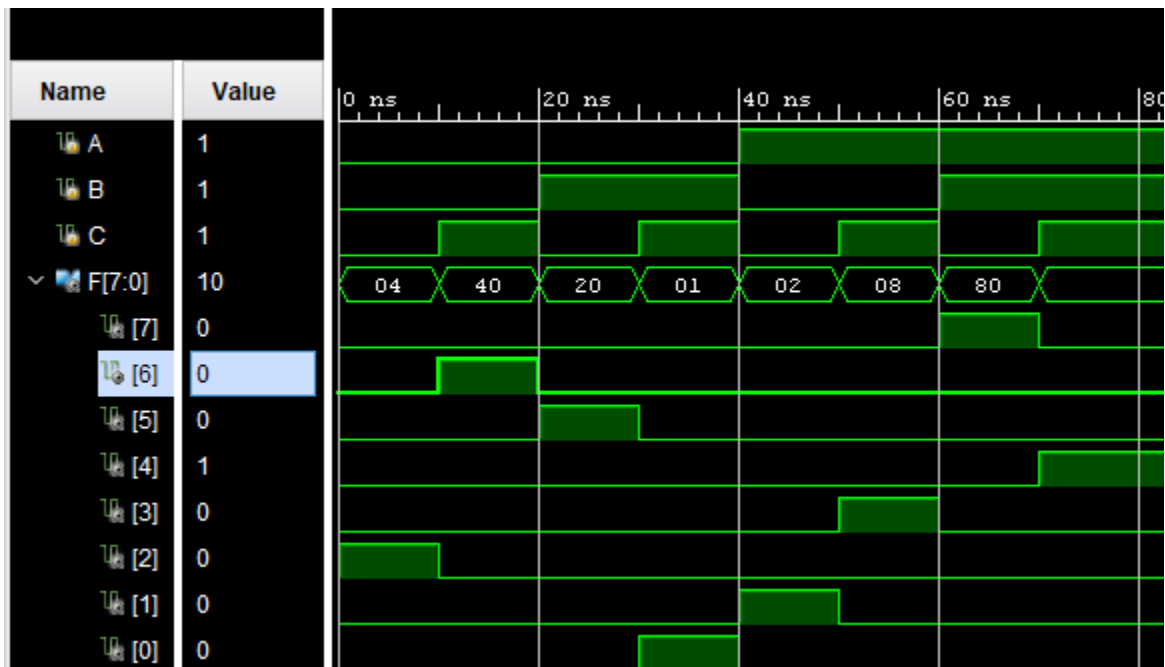
Table of I/O used on FPGA

In the constraints for the Artix-7 board I've specified the inputs and outputs of the board. From right to left on the switches, it goes C, then B, A, The output light from right to left is from F[0], F[1], ... F[7].

V14	U14	U15	W18	V19	U19	E19	U16	W16	V16	V17
F[7]	F[6]	F[5]	F[4]	F[3]	F[2]	F[1]	F[0]	A	B	C

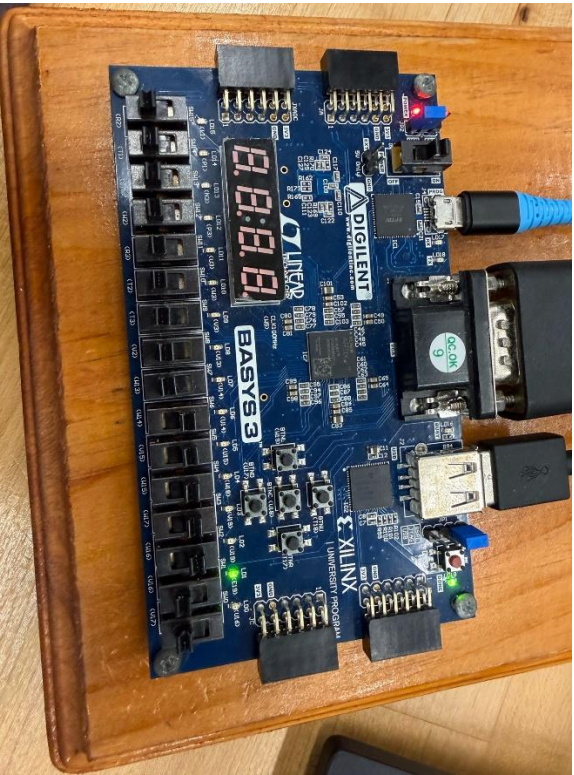
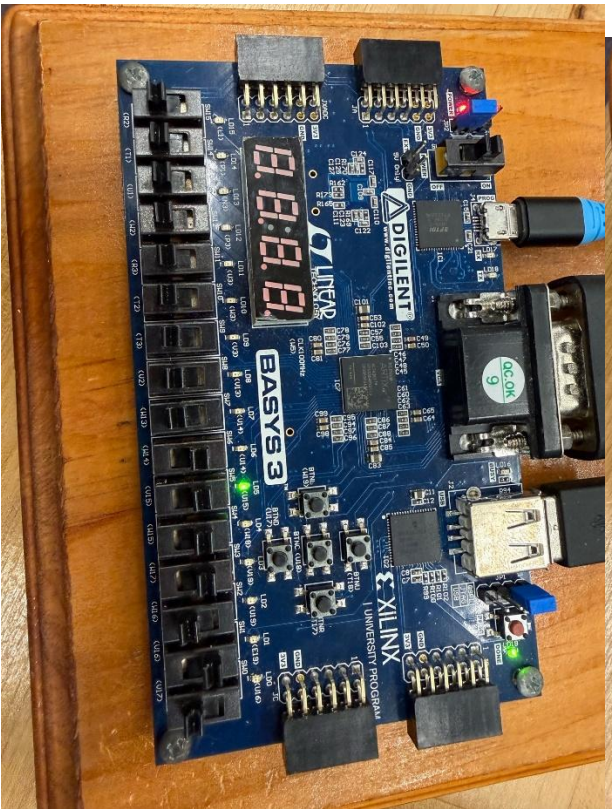
Screenshot of 3-8 mux waveform

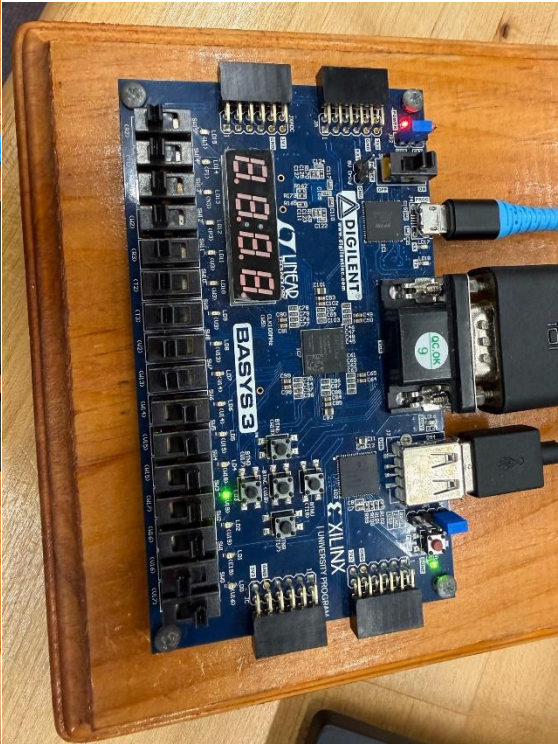
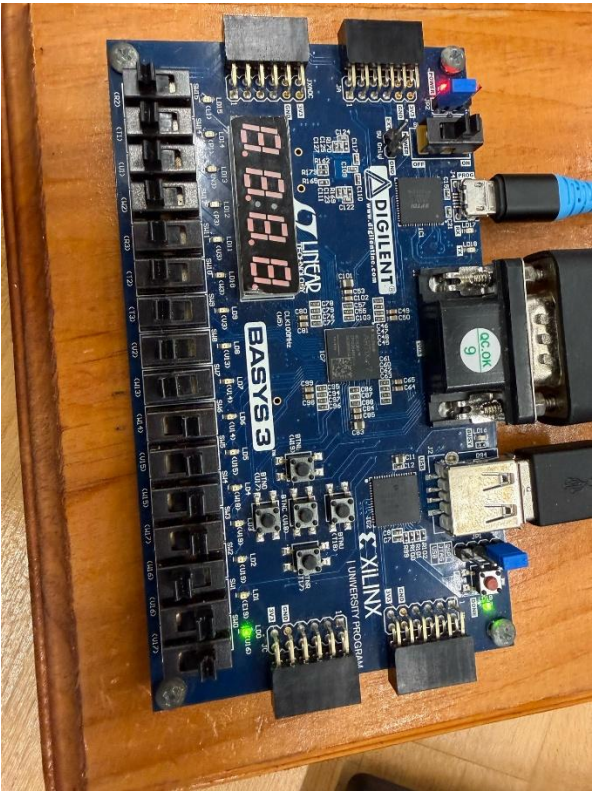
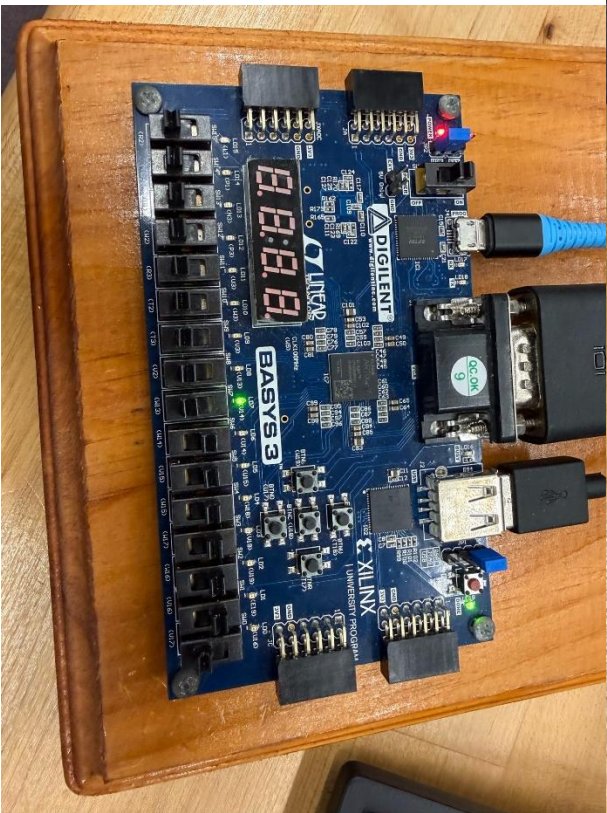
In this waveform, I've tested every iteration of A, B, and C. For F, there is only one instance where there is a 1, which changes through each iteration of A, B, C. For example, only F[0] is 1 when {a,b,c} = 011, or only F[2] = 1 when {a,b,c} = 000, and the rest of the bits would be 0.



Pictures of FPGA board

For the FPGA board, I've tested every iteration of A, B, and C, and the FPGA board showed only one output that is 1, for each iteration. This matches with waveform from earlier; only F[0] would turn on when {a,b,c} = 011, or only F[2] would turn on when {a,b,c} = 000, and the rest of the bits would be off.





Q&A:

What hardware component is referred to as many-to-one?

- Multiplexer

How many inputs would there be for a decoder if there were 32 outputs?

Show work.

- There would 5 inputs, because $2^5 \text{ inputs} = 32 \text{ outputs}$.

What are the names of the inverse designs of mux and decoder respectively?

- Inverse of mux(multiplexer) is demux (demultiplexer), and inverse of decoder is encoder.

What is the difference between a mux and a decoder and when would you use it? Explain

- A mux has selection lines, that determine which inputs are executed to the outputs, while decoders inputs determine which output to activate, and keep the others disabled. Mux would be used when the user needs to select one of many inputs, to send to a single destination, and the decoder would be used when you need to activate one of many outputs.

What are the anode and cathode used for on the 7-segment display?

- Anodes are positive terminals that are connected to a control pin, and cathode are negative terminals that are connected together, which is grounded.

What is the refresh period and frequency of the 7-segment display?

- Refresh period is the time taken for the led to update all digits once, and frequency is the refresh per second.

Test Plan:

Part 1 of mux: This testbench goes through every iteration of A, B, and S. For just part 1, the order is {S, B1, and A1}, whereas part 2 and 3 are in order of {A, B, ...}

```

1  | `timescale 1ns / 1ps
2  | //Lab5_Mux
3  | module mux_2to1_Sim();
4  |   reg A1, B1, S;
5  |   wire F;
6  |   integer i;
7  |
8  |   mux_2to1 UUT(A1, B1, S, F);
9  |   initial begin
10 |       for(i=0; i<2**3; i = i+1)begin
11 |           {S,B1,A1} = i;
12 |           #10;
13 |       end
14 |   end
15 | endmodule
16 |

```

Part 1 of decoder: This testbench goes through every iteration of A and B, which provides an output out of the 4 bits in F.

```

1  | `timescale 1ns / 1ps
2  |
3  |
4  | module decoder_2to4_Sim();
5  |   reg A2, B2;
6  |   wire [3:0] F;
7  |   integer i;
8  |
9  |   decoder_2to4 UUT(A2, B2, F);
10 |   initial begin
11 |       for(i=0; i<2**3; i = i+1)begin
12 |           {B2, A2} = i;
13 |           #10;
14 |       end
15 |   end
16 |
17 | endmodule
18 |

```

Part 2: This testbench doesn't go through every iteration of A, B, C and 8 bits in "in". Instead I tested 4 different arrangements of in: 00001111, 11110000, 01010101, and 00101100, and looped through {A, B, C} each time.

```

1  | `timescale 1ns / 1ps
2  |
3  | module mux_8to1_Sim();
4  |     reg A, B, C; // Select lines
5  |     reg [7:0] in;
6  |     wire F;
7  |     integer i;
8  |
9  |     mux_8to1 AUT(A,B,C,in, F);
10 | initial begin
11 |     in = 8'b00001111;
12 |     for(i = 0; i<2**3; i=i+1)begin
13 |         {A,B,C} = i;
14 |         #10;
15 |     end
16 |     in = 8'b11110000;
17 |     for(i = 0; i<2**3; i=i+1)begin
18 |         {A,B,C} = i;
19 |         #10;
20 |     end
21 |     in = 8'b01010101;
22 |     for(i = 0; i<2**3; i=i+1)begin
23 |         {A,B,C} = i;
24 |         #10;
25 |     end
26 |     in = 8'b00101100;
27 |     for(i = 0; i<2**3; i=i+1)begin
28 |         {A,B,C} = i;
29 |         #10;
30 |     end
31 | end
32 | endmodule
33 |

```

Part 3: This testbench goes through every iteration of A, B, and C. In both the waveform and FPGA board, have given that there is only one bit activated for the 8 bit output.

```

1  | `timescale 1ns / 1ps
2  |
3  | module decoder_3to8_sim();
4  |     reg A, B, C;
5  |     wire[7:0] F;
6  |     integer i;
7  |
8  |     decoder_3to8 AAT(A, B, C, F);
9  | initial begin
10 |     for(i=0; i<2**3; i = i+1)begin
11 |         {A, B, C} = i;
12 |         #10;
13 |     end
14 | end
15 |
16 | endmodule
17 |

```

Conclusion

This experiment was a success. The results were as expected for all parts of the experiment. I've had a few issues with setting up modified 8-1 mux, until I realized I could use minterms to help assign F with a specific "in" bit. Playing around with multiplexers and decoders to this extent using Vivado and FPGA board has furthered my understanding the logic, and how useful it can be.